

Experiments on Non-Linear Activation Functions (ReLU and Tanh) in CNN, using CIFAR-10 data

Wichita State University, Fall 2025

Submitted by Chandra Parasa

1. Dataset for the Experiment

The dataset for this experiment is CIFAR-10 (Krizhevsky, Nair, and Hinton,2009)¹, , which is a widely used benchmark in computer vision tasks. It contains 60,000 color images, each of size $32 \times 32 \times 3$ pixels, evenly distributed across 10 different categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

The dataset is divided into 50,000 training images and 10,000 test images. All images were normalized to have pixel values between 0 and 1, which helps stabilize training and improve numerical performance during optimization. CIFAR-10 is readily available on all major deep learning frameworks such as PyTorch and TensorFlow.

2. Creating a Four-Layer CNN

A four-layer convolutional neural network (CNN) was designed to extract progressively deeper features from input images and map them to class probabilities through a fully connected layer and a Softmax output. The architecture was kept identical for both activation functions to ensure fair comparison. Each convolutional layer applies filters that capture local spatial features such as edges, textures, and shapes. The pooling layers progressively reduce the spatial dimensions, helping to generalize better and reduce computational complexity. The fully connected layer integrates all learned features to classify the image into one of ten categories using the Softmax layer.

¹<http://www.cs.toronto.edu/~kriz/cifar.html>

Layer Type	Configuration	Activation
Convolutional (1)	3×3 filters, input channels = 3, output = 64	ReLU / Tanh
Convolutional (2)	3×3 filters, 64→128 channels	ReLU / Tanh
MaxPooling (1)	2×2 pooling window, reduces spatial size	—
Convolutional (3)	3×3 filters, 128→256 channels	ReLU / Tanh
Convolutional (4)	3×3 filters, 256→256 channels	ReLU / Tanh
MaxPooling (2)	2×2 pooling window	—
Fully Connected	256×8×8 → 512 neurons	ReLU / Tanh
Output Layer	512 → 10 classes	Softmax

Table 1: Architecture of the four-layer CNN used for CIFAR-10 classification.

3. Training Network until Training Error $\leq 25\%$

Both models were trained using the same optimizer, initialization scheme, and stopping conditions. The only differences were the activation function and learning rate, which were tuned separately for optimal performance with each activation.

Parameter	ReLU Model	Tanh Model
Optimizer	Stochastic Gradient Descent (SGD)	Stochastic Gradient Descent (SGD)
Learning Rate	0.05	0.004
Momentum	0.9	0.9
Batch Size	128	128
Weight Decay	0	0
Initialization	Normal($\mu = 0$, $\sigma = 0.01$)	Normal($\mu = 0$, $\sigma = 0.01$)
Gradient Clipping	1.0	1.0
Stopping Condition	Training error $\leq 25\%$	Training error $\leq 25\%$

Table 2: Training setup for both activation function experiments.

The models were trained using the cross-entropy loss function. Stochastic Gradient Descent (SGD) with momentum was chosen to accelerate convergence and smooth oscillations during training. Gradient clipping was used to prevent exploding gradients, ensuring numerical stability. Training was stopped immediately once the training error reached 25%.

4. Experiments with ReLU and Tanh Activations

Two experiments were performed using the same CNN architecture:

- **ReLU:** All hidden layers used ReLU activation, output layer used Softmax.
- **Tanh:** All hidden layers used Tanh activation, output layer used Softmax.

4.1. Training Performance Summary

Activation	Epochs to 25% Error	Avg. Time per Epoch (s)	Final Train Error
ReLU	5	~15.5	0.25
Tanh	29	~15.6	0.25

Table 3: Comparison of ReLU and Tanh training efficiency using GPU. ReLU reaches a 25% training error rate, about six times faster than Tanh.

4.2. Training Behavior

ReLU: The ReLU-based CNN showed a rapid drop in training error from 66.9% to below 25% within just five epochs. The model converged quickly because ReLU does not saturate for positive values, allowing gradients to flow effectively even in deep networks. The average time per epoch remained consistent at approximately 15.5 seconds. ReLU also demonstrated stable updates without significant oscillations in loss, suggesting efficient learning dynamics.

Tanh: The Tanh-based CNN started with a higher initial error (around 90.3%) and required 29 epochs to reach the same 25% error threshold. Although each epoch took nearly the same amount of time (around 15.6 seconds), the learning curve was smoother and more gradual. The slower convergence can be attributed to gradient saturation in the Tanh function, where gradients become very small for large input values. However, Tanh’s symmetric range (-1 to 1) helps center activations, which can improve stability and make learning less noisy in some contexts.

5. Time per Epoch, Visual Comparisons, and Discussion

5.1. Time per Epoch Graph

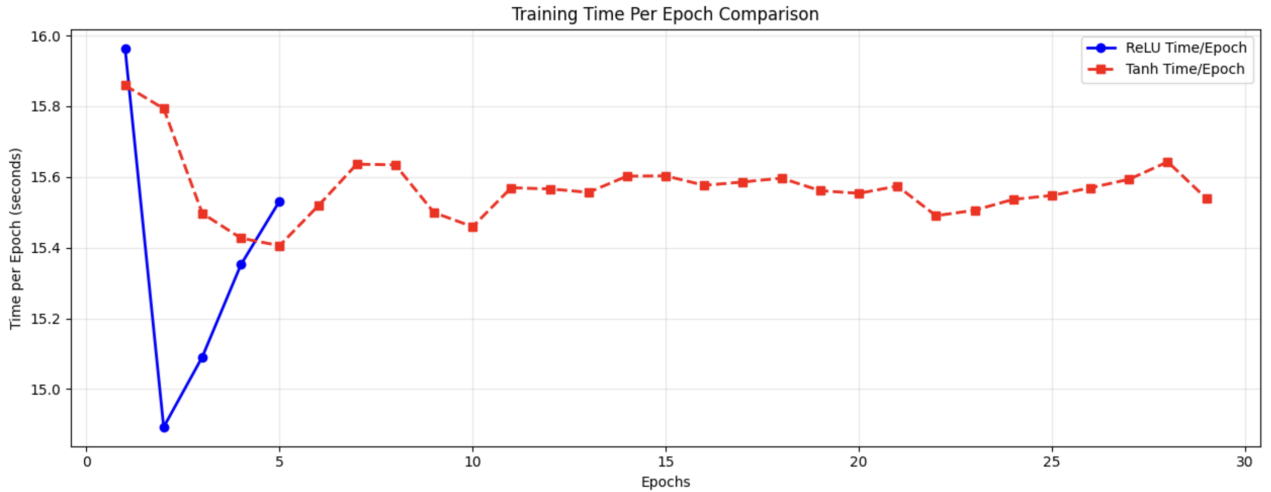


Figure 1: Training time comparison between ReLU and Tanh activations on CIFAR-10. X-axis: epoch number, Y-axis: time in seconds.

5.2. Training Error Comparison

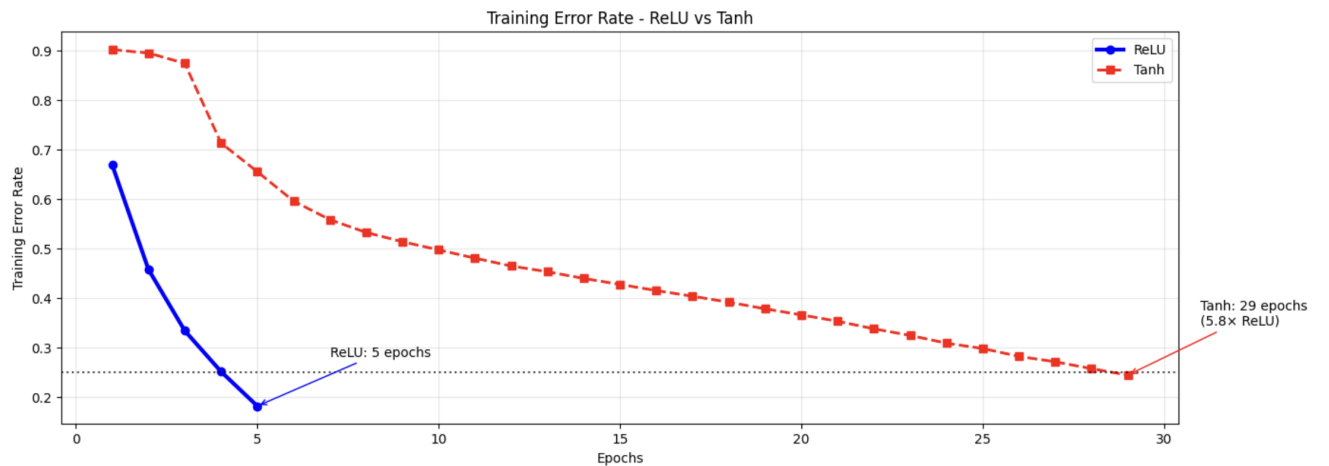


Figure 2: A four-layer CNN with ReLU (blue line) reaches a 25% training error rate on CIFAR-10 six times faster than Tanh (red line). ReLU with 5 epochs and Tanh with 29 epochs.

5.3. Discussion and Conclusion

The experiment highlights the impact of activation functions on both speed and behavior of CNN training.

- **ReLU** significantly improved convergence speed due to its linear, non-saturating gradient. This allows the model to learn deeper features efficiently and reach the

target accuracy in fewer epochs. However, ReLU can sometimes lead to “dead neurons,” where units stop updating if inputs fall in the negative range.

- **Tanh** provided smoother gradient updates and more stable learning behavior, though at the cost of speed. Its centered output (ranging between -1 and 1) reduces bias shifts across layers and can be beneficial for smaller or more sensitive architectures.

The results demonstrate that the activation function plays a crucial role in determining how quickly and efficiently a neural network learns. Under identical conditions, the CNN with ReLU activation achieved the target training accuracy almost six times faster than the model using Tanh. While Tanh delivered smoother learning and more stable updates, ReLU’s efficiency makes it a preferred choice for deep CNNs, particularly in large-scale image recognition tasks. Overall, ReLU activation offers a better balance between simplicity, computational efficiency, and learning speed for datasets like CIFAR-10.